

ITF23019: Parallel and Distributed Programming

Spring Semester, 2026

Lab7: Thread Synchronization 2

Submission Deadline: March 4th, 2026 23:59

You need at least 50 pts to pass this lab assignment.

This lab covers other synchronization mechanisms including **Conditional Variable**, **Semaphore**, and **Barrier** (in addition to **Monitor** and **Lock** in lab 6). Conditional Variables and Semaphores are different mechanisms to create and control a critical section while Barriers is used to synchronize multiple threads in a common point.

1 Conditional Variable

A thread can enter a critical section when a given condition happens. For example, in **Producer - Consumer** problem in previous lab, the producer enters the critical section when the buffer (queue) is not full and the consumer enters the critical section when the queue is not empty. On the contrary, when the queue is full, the producer is blocked and when the queue is empty, the consumer is blocked.

Conditional Variable is a synchronization mechanism that allows a thread to wait for a certain condition to become **True** to enter a critical section. In Conditional Variable, we use **ReentrantLock** to create a critical section and a variable, called a **conditional variable**, to create a condition. The purpose of the condition is to allow a thread to have control of a lock and check whether a condition is true or not. If the condition is not **True**, the thread is suspended until another thread wakes it up. In addition to **ReentrantLock**, the **Condition** interface which provides the mechanisms to suspend a thread (**await()** method) and to wake up a suspended thread (**signal()** or **signalAll()**) has to be used.

Implementation:

- **Step 1:** Use **Lock** to create a critical section

```
private final ReentrantLock lock;
```

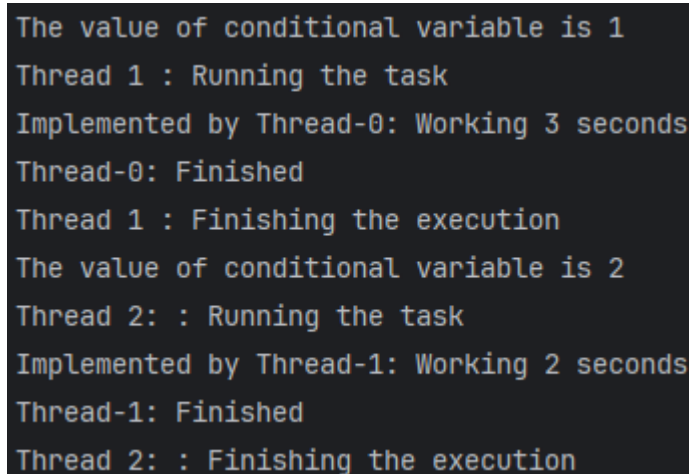
- **Step 2:** Use a variable to control the entering of the critical section

```
private final Condition condition; // use Condition interface which provides methods
                                   // to wake up and suspend a thread
private int turn; // conditional variable
```

The template for using a Conditional Variable is as follows:

```
private final ReentrantLock lock;
private final Condition condition; // use Condition interface which provides methods
// to wake up and suspend a thread
private int turn; // conditional variable
....
lock.lock();
// Check conditional variable
// Critical Section
// Change conditional variable
} catch (InterruptedException e) {
    e.printStackTrace();
} finally {
    lock.unlock();
}
.....
```

In the `conditionalVariable` project, the `TaskController` object synchronizes the execution of two threads: Thread 1 and Thread 2 using a conditional variable `turn`. Both threads will execute `CommonTask`. If `turn` is 1, Thread 1 will execute. If `turn` is 2, Thread 2 will execute. The expected output of the project is as follows:



```
The value of conditional variable is 1
Thread 1 : Running the task
Implemented by Thread-0: Working 3 seconds
Thread-0: Finished
Thread 1 : Finishing the execution
The value of conditional variable is 2
Thread 2: : Running the task
Implemented by Thread-1: Working 2 seconds
Thread-1: Finished
Thread 2: : Finishing the execution
```

Figure 1: Expected output of `conditionalVariable` project.

- Analyze the code in the `conditionalVariable` project yourself and run the code to see if you get the same output.

2 Semaphore

Similar to `Monitor` and `Lock`, `Semaphore` is used to implement a critical section but it is more flexible in sense that it allows multiple threads to enter the critical section simultaneously. This is done by setting the `counter` of the semaphore that keeps track of the number of threads that are currently accessing the shared resource. The initial value of the counter indicates how many threads can enter the critical section at the same time. In Figure 2, the value of counter is 2, meaning that two threads can enter the critical section and being executed at the same time. Other threads will be blocked. When one thread leaves the

critical section, another waiting thread can enter. If the value of counter is 1, **Semaphore** becomes **Lock**.

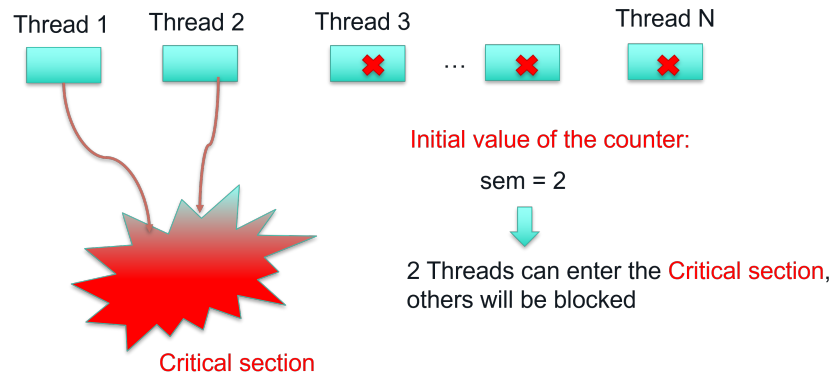


Figure 2: An illustration of Semaphore with counter equal 2.

In Java, the **Semaphore** is implemented in the `java.util.concurrent` package, specifically the **Semaphore** class. This class has two basic methods: `acquire()` and `release()`. A thread calls `acquire()` to ask for the permission to enter the critical section and calls `release()` to notify that it is leaving the critical section. The **Semaphore** keeps track of the number of threads which have been granted the permission. If a new `acquire()` is about to higher than the initial value of the counter, the calling thread is suspended until there is a room available. When a thread is leaving the critical section, it calls `release()` to notify a waiting thread of available space.

Implementation:

- **Step 1:** Create a Semaphore object by specifying the number of permits that are available for the shared resource. For example:

```
Semaphore semaphore = new Semaphore(5);
```

This means that 5 threads can enter the critical section at a time.

- **Step 2:** Each thread has to ask for the permission to the critical section by calling the `acquire()` method.

```
try {
    semaphore.acquire();
    // BLOCK OF CODE IN CRITICAL SECTION
} catch (InterruptedException e) {
    // Handle the exception
}
```

The `acquire()` method will block the thread until a permit is available.

- **Step 3:** Once the thread has finished accessing the critical section, it has to release the permit by calling the `release()` method:

```
semaphore.release();
```

The `release()` method increments the semaphore's permit count, allowing another thread to acquire the resource or to enter the critical section.

```
try {  
    semaphore.acquire();  
    CommonTask.Task();  
} catch (InterruptedException e) {  
    e.printStackTrace();  
} finally {  
    semaphore.release();  
}
```

Critical section

Figure 3: A task using semaphore to enter a critical section

The block of code between `acquire()` and `release()` is the critical section, as shown in Figure 3. Figure 4 - Figure 8 are different outputs of `semaphore` project with different values of `counter`.

```
Date: Wed Feb 25 13:02:14 CET 2026 pool-1-thread-1: Working 4 seconds  
Date: Wed Feb 25 13:02:18 CET 2026 pool-1-thread-3: Working 5 seconds  
Date: Wed Feb 25 13:02:23 CET 2026 pool-1-thread-3: Working 2 seconds  
Date: Wed Feb 25 13:02:25 CET 2026 pool-1-thread-4: Working 7 seconds  
Date: Wed Feb 25 13:02:32 CET 2026 pool-1-thread-2: Working 6 seconds  
Date: Wed Feb 25 13:02:38 CET 2026 pool-1-thread-5: Working 1 seconds  
Date: Wed Feb 25 13:02:39 CET 2026 pool-1-thread-6: Working 3 seconds  
Date: Wed Feb 25 13:02:42 CET 2026 pool-1-thread-7: Working 2 seconds  
Date: Wed Feb 25 13:02:44 CET 2026 pool-1-thread-8: Working 4 seconds  
Date: Wed Feb 25 13:02:48 CET 2026 pool-1-thread-1: Working 9 seconds
```

Figure 4: Output of the project when the initial value of the counter is 1. Semaphore is the same as Lock where only one thread can enter the critical section.

```
Date: Wed Feb 25 13:03:54 CET 2026 pool-1-thread-2: Working 8 seconds  
Date: Wed Feb 25 13:03:54 CET 2026 pool-1-thread-1: Working 6 seconds  
Date: Wed Feb 25 13:04:00 CET 2026 pool-1-thread-3: Working 4 seconds  
Date: Wed Feb 25 13:04:02 CET 2026 pool-1-thread-2: Working 3 seconds  
Date: Wed Feb 25 13:04:04 CET 2026 pool-1-thread-4: Working 2 seconds  
Date: Wed Feb 25 13:04:05 CET 2026 pool-1-thread-5: Working 6 seconds  
Date: Wed Feb 25 13:04:06 CET 2026 pool-1-thread-6: Working 2 seconds  
Date: Wed Feb 25 13:04:08 CET 2026 pool-1-thread-7: Working 3 seconds  
Date: Wed Feb 25 13:04:11 CET 2026 pool-1-thread-8: Working 5 seconds  
Date: Wed Feb 25 13:04:11 CET 2026 pool-1-thread-1: Working 1 seconds
```

Figure 5: Output when the initial value of the counter is 2 i.e, 2 threads can access the critical section at the same time.

```
Date: Wed Feb 25 13:09:33 CET 2026 pool-1-thread-3: Working 5 seconds
Date: Wed Feb 25 13:09:33 CET 2026 pool-1-thread-2: Working 3 seconds
Date: Wed Feb 25 13:09:33 CET 2026 pool-1-thread-1: Working 7 seconds
Date: Wed Feb 25 13:09:37 CET 2026 pool-1-thread-4: Working 6 seconds
Date: Wed Feb 25 13:09:39 CET 2026 pool-1-thread-3: Working 0 seconds
Date: Wed Feb 25 13:09:39 CET 2026 pool-1-thread-5: Working 5 seconds
Date: Wed Feb 25 13:09:41 CET 2026 pool-1-thread-6: Working 8 seconds
Date: Wed Feb 25 13:09:43 CET 2026 pool-1-thread-7: Working 7 seconds
Date: Wed Feb 25 13:09:44 CET 2026 pool-1-thread-8: Working 5 seconds
Date: Wed Feb 25 13:09:49 CET 2026 pool-1-thread-2: Working 9 seconds
```

Figure 6: Output when the initial value of the counter is 3 i.e, 3 threads can access the critical section at the same time.

```
Date: Wed Feb 25 13:13:44 CET 2026 pool-1-thread-5: Working 3 seconds
Date: Wed Feb 25 13:13:44 CET 2026 pool-1-thread-3: Working 8 seconds
Date: Wed Feb 25 13:13:44 CET 2026 pool-1-thread-1: Working 6 seconds
Date: Wed Feb 25 13:13:44 CET 2026 pool-1-thread-4: Working 8 seconds
Date: Wed Feb 25 13:13:44 CET 2026 pool-1-thread-2: Working 7 seconds
Date: Wed Feb 25 13:13:47 CET 2026 pool-1-thread-5: Working 9 seconds
Date: Wed Feb 25 13:13:50 CET 2026 pool-1-thread-6: Working 1 seconds
Date: Wed Feb 25 13:13:51 CET 2026 pool-1-thread-7: Working 8 seconds
Date: Wed Feb 25 13:13:51 CET 2026 pool-1-thread-8: Working 5 seconds
Date: Wed Feb 25 13:13:52 CET 2026 pool-1-thread-1: Working 1 seconds
```

Figure 7: Output when the initial value of the counter is 5 i.e, 5 threads can access the critical section at the same time.

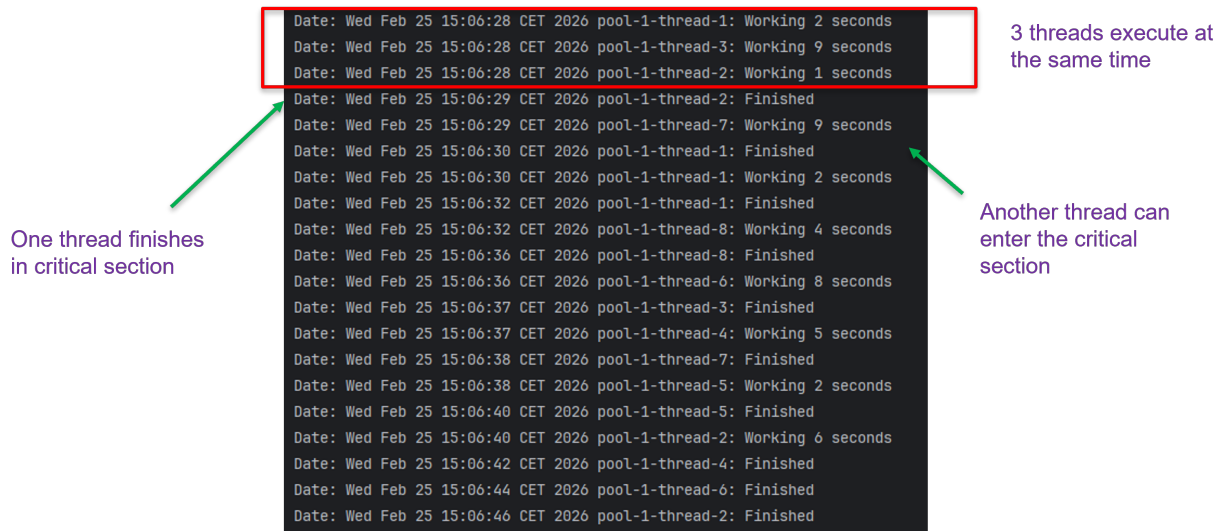


Figure 8: Another output when the initial value of the counter is 3.

When `counter` is 3, shown in Figure 8, the first three threads calling the `acquire()` can enter the critical section and other threads will be blocked. When one thread finishes its execution in the critical section and releases the semaphore with `release()`, as seen in the message `Date: Wed Feb 25 15:06:29 CET 2026 pool-1-thread-2: Finished`, another thread can enter the critical section, seen in the `Date: Wed Feb 25 15:06:29 CET 2026 pool-1-thread-7: Working 9 seconds`. The procedure is repeated: when one thread finishes, another calling thread will get access to the critical section.

- Analyze the code in the `semaphores` project yourself and run the code to see if you get the same output.
- Add more code in the critical section and run the program to see how different threads can access the critical section at the same time.

3 Barrier

Barrier is a synchronization mechanism that allows multiple threads to synchronize with each other at a specific point, called a **barrier**. These threads will wait at the barrier until all of them have arrived, at which point they can proceed with their tasks.

In Java, the barrier synchronization is implemented through the `java.util.concurrent` package, specifically the `CyclicBarrier` class. This class is initialized with an integer number indicating the number of threads that will be synchronized at a common point. When one of these threads arrives at the common point, it calls the `await()` method to wait for the other threads. When a thread calls `await()` method, the thread is blocked until all the other threads have arrived at this point. When the last thread calls the `await()` method, it wakes up all the threads that are waiting and continues with its job. `CyclicBarrier` class also allows to pass an additional `Runnable` object as an initialization parameter. The `CyclicBarrier` class then executes this object as a thread when all the threads arrive at the common point.

Implementation:

- **Step 1:** Create a `CyclicBarrier` object and specify the number of threads that will participate in the synchronization. For example:

```
CyclicBarrier barrier = new CyclicBarrier(5);
```

This command creates a barrier that will wait for five threads to arrive before allowing them to proceed further.

- **Step 2:** Each thread calls the `await()` method on the barrier object to wait for the other threads to arrive:

```
try {
    barrier.await();
} catch (InterruptedException | BrokenBarrierException e) {
    // Handle the exception
}
```

The `await()` method will block the thread until all the threads have arrived at the barrier.

- **Step 3:** Once all the threads have arrived, the barrier will be broken and the threads can proceed with their tasks. Note that the `CyclicBarrier` object can be reused after the threads have passed through it. This means that you can reuse the same barrier for multiple synchronization points in your program.

The `barrier` project illustrates the use of barrier synchronization mechanism. The `Task` class implements a barrier task, which will be executed by a thread later on. After doing a common job, i.e, sleep for 100ms, each thread has to wait for other threads to finish sleeping by calling the `await()` method. This common point is called Phase 1. After all 10 threads finish executing 10 barrier tasks, the threads move to Phase 2, shown in Figure 9.

```
public void run() {
    System.out.println(Thread.currentThread().getName()+" Phase 1");
    CommonTask.doTask();
    try {
        barrier.await();
    } catch (InterruptedException e) {
        e.printStackTrace();
    } catch (BrokenBarrierException e) {
        e.printStackTrace();
    }
    System.out.println(Thread.currentThread().getName()+" Phase 2");
}
```

Figure 9: A code snippet of `barrier` project.

The project also implements another `Runnable` object that will be executed by `CyclicBarrier` when all the tasks have executed the `await()` method.

```
public class FinishBarrierTask implements Runnable {
    public void run() {
        System.out.println("FinishBarrierTask: All the tasks have finished");
    }
}
```

In the Main class, the main() method creates 10 barrier tasks. The CyclicBarrier is initialized by 10 which is the number of tasks we want to synchronize. These tasks will be executed in the Executor framework after they call await().

```
public static void main(String[] args) {
    CyclicBarrier barrier=new CyclicBarrier(10,new FinishBarrierTask());
    ThreadPoolExecutor executor=(ThreadPoolExecutor)Executors.newCachedThreadPool();
    for (int i=0; i<10; i++) {
        executor.execute(new BarrierTask(barrier));
    }
    executor.shutdown();
    try {
        executor.awaitTermination(1, TimeUnit.DAYS);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}
```

The output of the program is as follows:

```
pool-1-thread-1: Phase 1
pool-1-thread-3: Phase 1
pool-1-thread-2: Phase 1
pool-1-thread-4: Phase 1
pool-1-thread-5: Phase 1
pool-1-thread-6: Phase 1
pool-1-thread-7: Phase 1
pool-1-thread-8: Phase 1
pool-1-thread-9: Phase 1
pool-1-thread-10: Phase 1
pool-1-thread-1: Sleeping for a duration of 100 ms
pool-1-thread-9: Sleeping for a duration of 100 ms
pool-1-thread-6: Sleeping for a duration of 100 ms
pool-1-thread-10: Sleeping for a duration of 100 ms
pool-1-thread-8: Sleeping for a duration of 100 ms
pool-1-thread-7: Sleeping for a duration of 100 ms
pool-1-thread-5: Sleeping for a duration of 100 ms
pool-1-thread-2: Sleeping for a duration of 100 ms
pool-1-thread-3: Sleeping for a duration of 100 ms
pool-1-thread-4: Sleeping for a duration of 100 ms
FinishFirstPhase: All the tasks have finished sleeping
FinishFirstPhase: Now moving to the next Phase
pool-1-thread-4: Phase 2
pool-1-thread-1: Phase 2
pool-1-thread-9: Phase 2
pool-1-thread-6: Phase 2
pool-1-thread-10: Phase 2
pool-1-thread-8: Phase 2
pool-1-thread-7: Phase 2
pool-1-thread-5: Phase 2
pool-1-thread-2: Phase 2
pool-1-thread-3: Phase 2
```

Barrier synchronization mechanism is useful in scenarios where multiple threads need to perform some

related tasks, and these tasks cannot be performed until all the threads have completed their individual tasks. The barrier synchronization mechanism ensures that all the threads are synchronized and can proceed with their tasks together.

4 Exercises

4.1 Exercise 1 (30 points)

The program below is designed to create two threads: thread1 and thread2, each of which prints a message three times. Thread thread1 prints the message “Debugging begins in 3... 2... 1...”, while thread2 prints the message “Welcome to the world of programming.” However, several blocks of code, marked with `// TODO keywords`, are missing.

```
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;

public class SimpleConditionExample {
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition condition = lock.newCondition();
    private int turn = 2;
    public void doTaskA() {
        lock.lock();
        try {
            // TODO: Your code for synchronization here
            // ...
            System.out.print("Debugging begins in 3... 2... 1...\n");
            // TODO: Your code for synchronization here
            // ...
        } catch (InterruptedException e) {
            e.printStackTrace();
        } finally {
            lock.unlock();
        }
    }
    public void doTaskB() {
        lock.lock();
        try {
            // TODO: Your code for synchronization here
            // ...
            System.out.print("Welcome to the world of programming. ");
            // TODO: Your code for synchronization here
            // ...
        } catch (InterruptedException e) {
            e.printStackTrace();
        } finally {
            lock.unlock();
        }
    }
}

static class TaskA implements Runnable {
    private final SimpleConditionExample controller;
```

```

    public TaskA(SimpleConditionExample controller) {
        this.controller = controller;
    }
    public void run() {
        // TODO: Your code to implement TaskA
        // which will be executed by one thread in main()
    }
}
static class TaskB implements Runnable {
    private final SimpleConditionExample controller;
    public TaskB(SimpleConditionExample controller) {
        this.controller = controller;
    }
    public void run() {
        // TODO: Your code to implement TaskA
        // which will be executed by one thread in main()
    }
public static void main(String[] args) throws InterruptedException {
    SimpleConditionExample controller = new SimpleConditionExample();
    // TODO: your code to create thread1 and thread2
    // ...
    // TODO: your code to start and join newly created threads
    // ...
    // TODO: your code to print the message:
    // May your code compile on the first try (eventually).
    // ...
}
}

```

Complete the code while ensuring that the following requirements are met:

- In Main class, create thread1 and thread2 in sequence. Thread1 executes `doTaskA()` and Thread2 executes `doTaskB()`.
- Use Conditional Variables to synchronize thread1 and thread2 so that the output of the program is as follows:

```

Welcome to the world of programming. Debugging begins in 3... 2... 1...
Welcome to the world of programming. Debugging begins in 3... 2... 1...
Welcome to the world of programming. Debugging begins in 3... 2... 1...
May your code compile on the first try (eventually).

```

4.2 Exercise 2 (30 points)

Do the same thing with Exercise1. However, use **Semaphores** to synchronize two threads instead of using Conditional Variables.

4.3 Exercise 3 (40 points)

A project group consisting of five members holds weekly meetings in a physical room. Before the meeting, members may arrive at different times. The meeting room is equipped with a sensor that counts the number of participants entering the meeting room. The system is designed to automatically start the meeting once the sensor detects that all five participants have arrived.

The project `Exercise3` simulates this system, however, several blocks of code are missing. Your task in this Exercise is to complete the implementation by using `Barrier` to synchronize threads so that the meeting will start only when all participants have arrived. The missing code in the project is marked with `// TODO` keywords. The expected output of the project is similar to the followings:

```
Participant 0 has arrived.
Project Meeting: Waiting for 4 remaining participants.
Participant 3 has arrived.
Project Meeting: Waiting for 3 remaining participants.
Participant 1 has arrived.
Project Meeting: Waiting for 2 remaining participants.
Participant 2 has arrived.
Project Meeting: Waiting for 1 remaining participants.
Participant 4 has arrived.
The remaning participant has arrived. All the participants are there!
Project Meeting: Initialization: 5 participants.
Project Meeting: Let's start the meeting...
```

5 What To Submit

Complete the the exercises in this assignment. Then, put your codes inside the `lab7` directory of your repository; we will test your code. Commit your changes and remember to check the GitHub website to make sure all files have been submitted. This lab does not ask for report but you can make a report if you like.

6 References

1. González, Javier Fernández. *Mastering Concurrency Programming with Java 9*. Packt Publishing Ltd, 2017.